

Lab 1: 2D Drawing

王唐欣宇 — 2400017808

September 17, 2025

1 Task 1: Image Dithering

1.1 Uniform Random

I add uniform noise $\epsilon \sim \mathcal{U}(-0.5, 0.5)$ to each pixel before thresholding, implemented using `std::minstd_rand` and `std::uniform_real_distribution<float>`. The generator seed is fixed to 0 for determinism. The result is shown in Figure 1a.

1.2 Blue-Noise Random

I compute the mean of the RGB channels of `noise` and subtract it, making the noise have zero mean. For each pixel $c(x, y)$, I add the adjusted noise and threshold at 0.5. The result is shown in Figure 1b.

1.3 Ordered Dithering

I use a 3×3 Bayer-like threshold matrix to perform ordered dithering. For each input pixel $c(x, y)$, I compare its normalized RGB channels with the corresponding threshold in the matrix, and assign the output to either 0 or 1 accordingly. The matrix is tiled over the entire image by repeating it every 3 pixels in both horizontal and vertical directions. The result is shown in Figure 1c.

1.4 Error Diffusion

I implement error diffusion dithering following the standard procedure described in the textbook. For each pixel, the RGB channels are thresholded at 0.5, and the quantization error is propagated to neighboring pixels using the Floyd–Steinberg weights. The result is shown in Figure 1d.

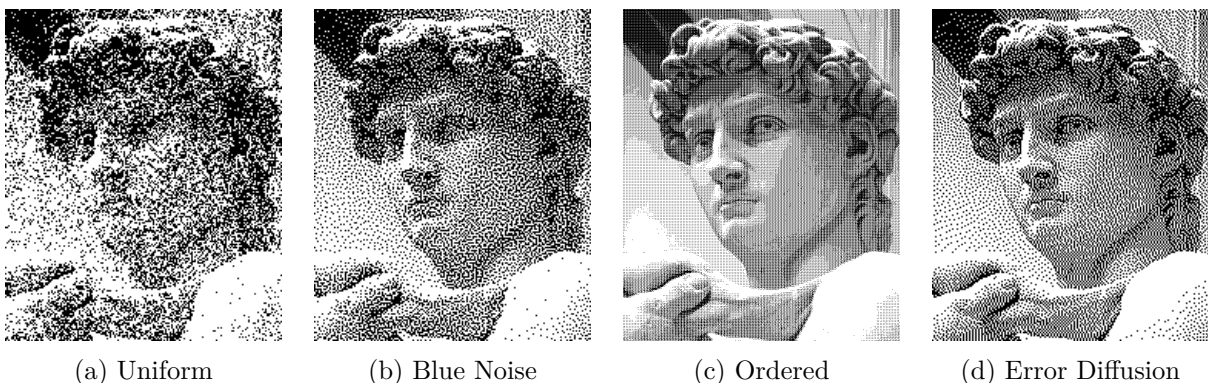


Figure 1: The Result of Task 1.

2 Task 2: Image Filtering

2.1 Box Blur (3x3)

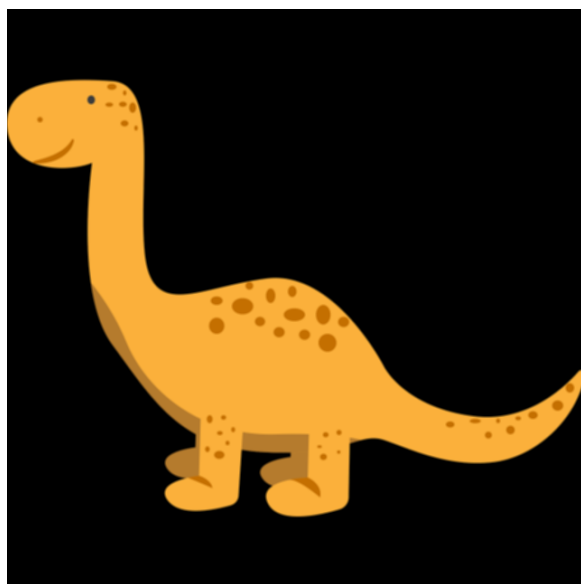
I apply a normalized 3×3 box kernel. Border handling uses zero padding. The result is shown in Figure 2.

2.2 Edge Detection (Sobel)

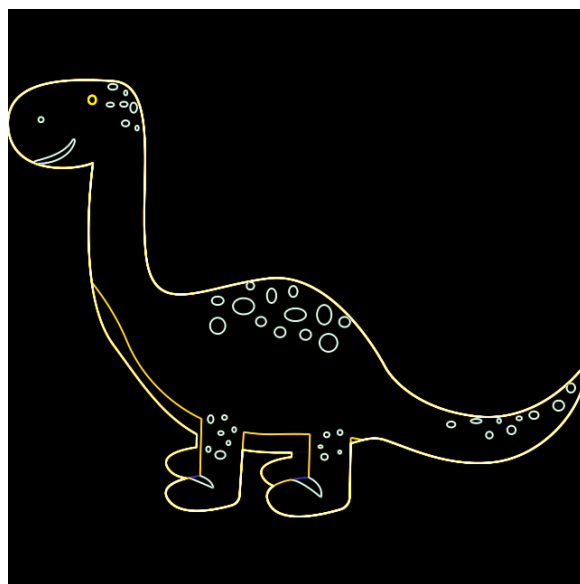
I convolve with Sobel kernels G_x and G_y , and take the magnitude per channel:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}.$$

Magnitude is computed channel-wise as $\sqrt{G_x^2 + G_y^2}$. The result is shown in Figure 2.



(a) Blur Result.



(b) Edge Result.

Figure 2: The Result of Task 2.

3 Task 3: Image Inpainting

This section only needs me to fill the margin value in the blank area. Here is my understanding of the Poisson Editing methods.

we use $g = f - f^*$, where f^* is pixels value in front image, f is what we want to calculate. if we calculate the g , we can add f^* to f , which is the last code the template provide.

As the template already provide the code to calculate g . We only need to fill the margin value. we need $f(x, y)$ in margin be same as background image. So we can calculate the $g(x, y)$ in margin, which is how I implement the code.

The result is shown in Figure 3.

4 Task 4: Line Drawing

The method I implement in the code is slightly different from the typical algorithm in the class. I use a value error to represent the distance between the ideal line and the rasterized line,



Figure 3: Result of Image Inpainting.

both in x and y direction. And after drawing each pixel, I update the error and decide whether to step in x or y direction based on the error.

The detailed math is under:

It's also normal to assume the start is from (0, 0) to (dx, dy).

$$F(x, y) = dy * x - dx * y$$

now step is (x_{now}, y_{now}) , which is integer, we want to decide the next step is $(x_{now} + 1, y_{now})$ or $(x_{now}, y_{now} + 1)$ or $(x_{now} + 1, y_{now} + 1)$. which means the F must be as close to 0 as possible. and if $F(x, y) < (-2 / dy)$ that we need to step x to $x + 1$, if $F(x, y) > (2 / dx)$ we need to step y to $y + 1$, else we can step both x and y to $x + 1$ and $y + 1$. that was what I implement in the code. The result is shown in Figure 4.

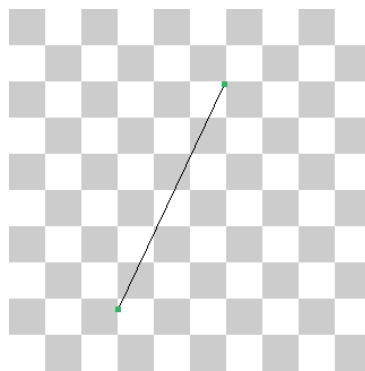


Figure 4: Line drawing.

5 Task 5: Triangle Drawing

I use a lambda function to compute the edge function for each edge of the triangle row. For each pixel in the bounding box of the triangle, I evaluate the edge functions to determine if the pixel is inside the triangle. If it is, I set the pixel color accordingly.

The result is shown in Figure 5.

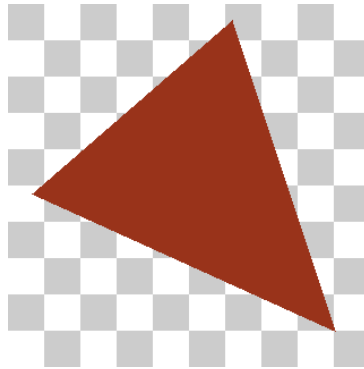


Figure 5: Triangle drawing.

6 Task 6: Image Supersampling

First I use bilinear interpolation to sample the image in $\text{size}_{\text{output}} \times \text{rate}$, and then I use a box filter to downsample the image to the output size. The result is shown in Figure 6.



(a) Origin Photo.



(b) Photo in Rate 5.



(c) Photo in Rate 12.

Figure 6: The Result of Task 6.

7 Task 7: Bezier Curve

I implement the de Casteljau's algorithm to compute the Bezier curve points. The algorithm recursively interpolates between control points to generate points on the curve. The result is shown in Figure 7.

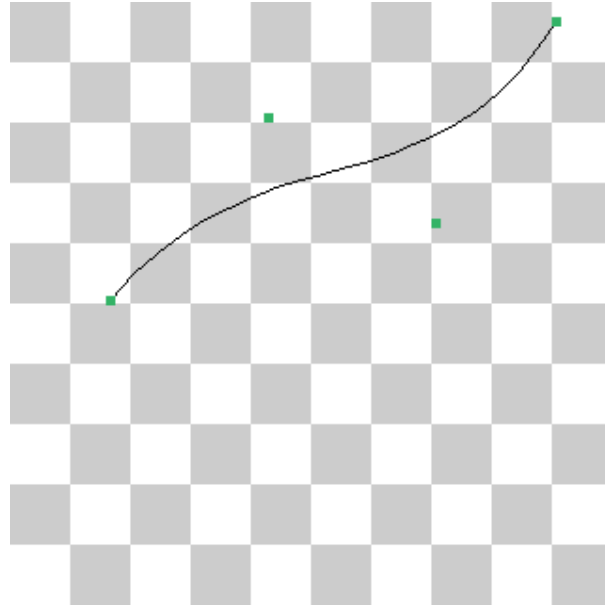


Figure 7: Bezier curve example (placeholder image).